



Wind Turbine Fault Detection

1 Introduction and Problem Context

Wind turbines are critical assets in renewable energy systems, yet unexpected failures can lead to costly downtime and lost power generation. Traditional maintenance strategies are often reactive or schedule-based and do not fully utilize the rich data available from Supervisory Control and Data Acquisition (SCADA) systems.

This project aims to develop supervised machine learning models to predict whether a turbine will experience a failure within a short future window (e.g., 24–72 hours). The problem is formulated as a binary classification task, where each timestamp is labeled based on whether a failure occurs within a defined horizon. This formulation emphasizes early detection of degradation rather than precise prediction of failure timing.

The dataset consists of multivariate, time-stamped SCADA measurements across multiple turbines and wind farms, including environmental, mechanical, electrical, and thermal variables. While well-suited for predictive modeling, it presents several challenges, including severe class imbalance, missing data, and variability across turbines. These characteristics make the problem inherently difficult and limit the achievable predictive performance.

The project has evolved into the development of an end-to-end predictive maintenance pipeline, including data preprocessing, feature engineering, labeling, modeling, and evaluation aligned with operational decision-making. As the project has progressed, greater emphasis has been placed on temporal modeling, labeling strategy, and aligning model performance with operational usability.

2 Data Understanding and Exploratory Analysis

This section summarizes key findings from exploratory data analysis (EDA), with a focus on identifying patterns, inconsistencies, and challenges that directly informed feature engineering and modeling decisions.

2.1 Failure Behavior Emerges Through Gradual Degradation

Exploratory analysis revealed that turbine failures are not preceded by abrupt changes in individual sensor readings, but rather by gradual shifts in system behavior over time. These changes include increasing vibration levels, rising temperatures, and growing instability in operational signals.

This observation indicates that failure is better characterized as a degradation process rather than a discrete event. As a result, capturing temporal dynamics—rather than relying on point-in-time measurements—is critical for predictive modeling.

2.2 Cross-Farm Inconsistencies Require Harmonization

The dataset includes turbines from multiple wind farms, each with variations in sensor availability, naming conventions, and value distributions. Even when sensors represent the same physical quantity, their recorded values may differ due to calibration or environmental factors.

These inconsistencies pose a challenge for building a unified model that generalizes across turbines. To address this, a sensor harmonization strategy was required to align feature definitions and ensure consistency across farms.

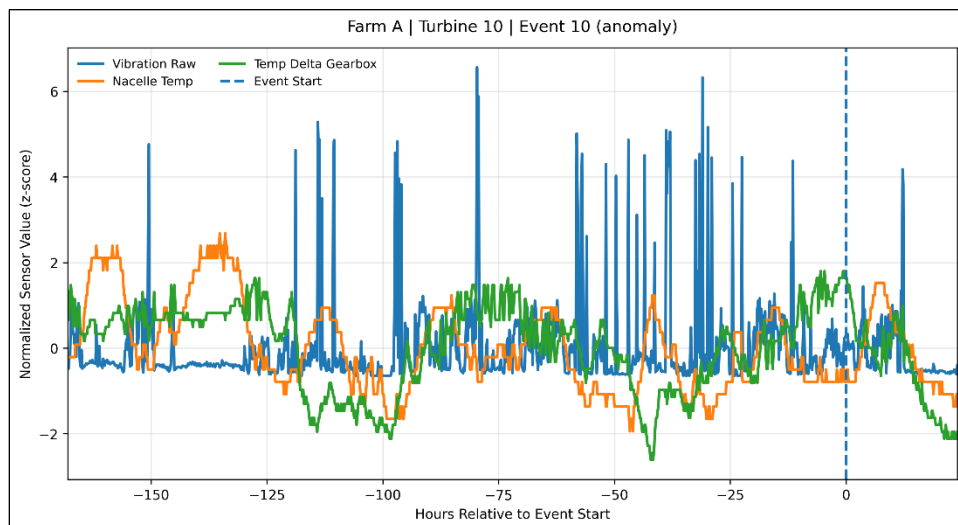
2.3 Missingness and Data Quality Vary Across Sensors

Data completeness varies significantly across features and turbines. Some sensors exhibit near-complete coverage, while others contain substantial missing values or intermittent gaps.

Rather than applying a uniform approach, features with excessive missingness were excluded, while others were retained and handled in a way that preserves temporal continuity. This step was necessary to balance data quality with feature coverage.



2.4 Early Warning Signals Are Present but Not Precisely Localized



EDA indicates that failure-related signals often appear several days prior to the actual event. However, these signals are not confined to a sharply defined time window and may fluctuate in intensity over time.

This reinforces the idea that failure prediction involves identifying a broader degradation process rather than pinpointing a precise moment of failure. It also introduces ambiguity in defining the optimal prediction horizon, motivating the evaluation of multiple labeling windows in later sections.

2.5 Implications for Modeling

These findings directly informed the design of the modeling pipeline:

- The need to capture temporal dynamics motivated the use of rolling, lag, and rate-of-change features.
- Sensor inconsistencies required harmonization and careful feature selection.
- Diffuse failure signals influenced labeling strategy and evaluation design.
- Data quality variability guided feature inclusion and preprocessing decisions.

Together, these insights establish the foundation for the feature engineering and modeling approaches described in subsequent sections.

3 Feature Engineering Strategy

This section describes the feature engineering approach used to transform raw SCADA measurements into representations that capture the temporal and physical dynamics of turbine behavior. The design of these features was guided directly by insights from exploratory analysis, particularly the observation that failure signals emerge through gradual degradation rather than abrupt changes.

3.1 Emphasis on Temporal Representations

Exploratory analysis showed that failure-related signals are distributed over time, with gradual changes in vibration, temperature, and operational stability preceding failure events. As a result, the feature engineering strategy shifted away from static sensor values toward representations that capture temporal behavior.

To support this, features were constructed using rolling statistics, lagged values, and rate-of-change measures. These transformations allow the model to capture trends, short-term fluctuations, and directional changes in system behavior, which are more indicative of degradation than instantaneous measurements.



3.2 Capturing Physically Meaningful Behavior

Feature design was guided by known physical mechanisms of turbine operation and failure. Mechanical signals such as vibration, thermal indicators such as component temperatures, and measures of operational instability were prioritized, as these are directly related to system degradation.

In addition to raw sensor values, derived features were introduced to better capture dynamic behavior. For example, temperature deltas highlight abnormal heating patterns, while volatility-based features quantify instability in control signals. These transformations improve the model's ability to detect subtle but meaningful deviations from normal operation.

3.3 Managing Feature Consistency Across Turbines

Given the variability in sensor availability and naming across wind farms, feature engineering also required ensuring consistency across turbines. This was addressed through a harmonization process that aligns equivalent sensors and prioritizes features that are broadly available.

Where possible, derived features were favored over raw signals, as they tend to be more robust to differences in scale and distribution. This approach helps improve generalization and reduces the risk of models learning farm-specific artifacts.

3.4 Controlling Feature Complexity and Redundancy

The introduction of temporal and derived features significantly expands the feature space, increasing the risk of redundancy and overfitting. To manage this, feature selection was guided by both data quality and interpretability.

Highly correlated or redundant features were reduced where appropriate, and features with excessive missingness were excluded. This resulted in a feature set that balances predictive richness with stability and computational efficiency.

3.5 Implications for Modeling

The resulting feature set enables the model to capture the key characteristics of turbine degradation, including gradual trends, instability, and dynamic changes in system behavior. These features provide the foundation for the modeling framework described in the next section.

More broadly, the feature engineering process reinforces a central theme of the project: that effective failure prediction depends on representing how system behavior evolves over time, rather than relying solely on static measurements.

4 Data Pipeline and Labeling Strategy

This section describes the data pipeline used to transform raw SCADA time-series data into a structured dataset suitable for supervised learning. Emphasis is placed on the labeling strategy and temporal design decisions required to ensure realistic and leakage-free model evaluation.

4.1 Pipeline Overview

The data pipeline integrates preprocessing, feature engineering, labeling, and dataset splitting into a unified workflow. All operations are performed at the turbine level while preserving chronological ordering, ensuring that temporal dependencies are maintained throughout the process.

This structure supports reproducibility and ensures that all transformations are applied consistently across training, validation, and test data.

4.2 Defining the Prediction Task

The objective of the model is to predict whether a turbine will experience a failure within a specified future time window. This is implemented by assigning a binary label to each timestamp based on whether a failure event occurs within the defined horizon.

Rather than predicting the exact moment of failure, this formulation focuses on identifying early warning signals that indicate ongoing degradation. This aligns the modeling task with practical maintenance objectives, where advance notice is more valuable than precise timing.



4.3 Labeling Strategy and Prediction Windows

To evaluate how prediction horizon impacts model behavior, multiple labeling windows were considered, including 24-hour, 48-hour, and 72-hour horizons.

Each timestamp is labeled as positive if a failure event occurs within the corresponding future window, and negative otherwise. This approach allows the model to learn patterns associated with pre-failure conditions while enabling comparison across different operational definitions of “early warning.”

As shown in later sections, model performance is relatively consistent across these windows, suggesting that failure signals are not confined to a narrowly defined time boundary.

4.4 Buffer Zones and Leakage Prevention

A key challenge in time-series labeling is preventing ambiguity and data leakage near failure events. To address this, buffer zones were introduced around each event.

Buffer periods remove data immediately before and after the failure event, where system behavior may be unstable or partially degraded. This ensures that:

- Positive labels reflect meaningful pre-failure conditions rather than immediate pre-event artifacts.
- Negative samples are not contaminated by post-event effects.
- Temporal leakage is minimized.

This design creates a clear separation between normal operation, pre-failure periods, and excluded transition regions, resulting in more reliable model training and evaluation.

4.5 Temporal Splitting and Evaluation Design

To ensure realistic performance assessment, the dataset is split in a way that preserves temporal ordering. Training, validation, and test sets are constructed chronologically, preventing future information from leaking into the training process.

This approach reflects real-world deployment conditions, where models are trained on historical data and applied to future observations. It also ensures that evaluation metrics provide a meaningful estimate of generalization performance.

4.6 Implications for Modeling

The design of the data pipeline and labeling strategy has a direct impact on model behavior. In particular, the use of temporal windows and buffer zones shifts the problem from predicting exact failure timing to identifying broader degradation patterns.

This reinforces a central theme of the project: that data design decisions—especially labeling and temporal structure—are as critical as model selection in determining overall performance.

5 Modeling Framework

This section describes the modeling approach used to evaluate the predictive capability of the engineered features and labeled dataset. The framework was designed to support consistent comparison across models while addressing key challenges such as class imbalance and threshold sensitivity.

5.1 Model Selection

Three classification models were evaluated to represent different modeling approaches: Logistic Regression, Random Forest, and XGBoost.

Logistic Regression serves as a linear baseline, providing a reference point for understanding model behavior under simplified assumptions. Random Forest introduces nonlinear modeling through ensemble decision trees, while XGBoost extends this approach using gradient boosting to capture more complex interactions and patterns.

This combination allows for comparison between linear and nonlinear models and provides insight into how different modeling strategies respond to the temporal and engineered feature set.



5.2 Pipeline Design and Consistency

To ensure reproducibility and prevent data leakage, all models were implemented within a unified pipeline framework. This integrates preprocessing steps, such as feature scaling where appropriate, directly into the modeling process.

By embedding preprocessing within the pipeline, transformations are learned exclusively from training data and applied consistently to validation and test sets. This design ensures that evaluation results reflect realistic deployment conditions and are not influenced by unintended information leakage.

5.3 Handling Class Imbalance

The dataset exhibits significant class imbalance, with failure events representing a small fraction of observations. Without mitigation, this imbalance can lead to models that prioritize the majority class and fail to detect meaningful failure patterns.

To address this, class weighting techniques were applied across models to increase the influence of failure events during training. This approach encourages models to identify minority class patterns while maintaining overall stability.

5.4 Evaluation Metrics and Threshold Selection

Given the imbalanced nature of the problem, model evaluation focuses on metrics that balance precision and recall. F1 score and Precision-Recall analysis were used as primary evaluation tools, as they provide a more meaningful assessment of performance than accuracy in this context.

In addition, threshold optimization was introduced to address the mismatch between predicted probabilities and true event likelihood. Rather than using a default threshold, classification thresholds were selected based on validation performance to achieve a more effective balance between detecting failures and limiting false positives.

5.5 Implications for Model Behavior

The modeling framework reflects a shift from purely statistical evaluation toward operationally meaningful performance. In particular, the combination of class weighting, appropriate metrics, and threshold optimization ensures that model outputs are evaluated in terms of their practical utility.

This approach highlights the importance of aligning modeling decisions with real-world constraints, reinforcing that effective performance depends not only on model selection but also on how predictions are interpreted and applied.

6 Preliminary Results

This section presents the results of applying the modeling framework to the engineered dataset. The focus is on understanding model performance, comparing prediction windows, and interpreting model behavior in the context of turbine degradation.

6.1 Model Performance Across Prediction Windows

Model performance was evaluated across three labeling strategies: 24-hour, 48-hour, and 72-hour prediction windows. This comparison provides insight into how prediction horizon impacts model behavior.

These results highlight how the choice of prediction window influences model performance, while also revealing that the underlying predictive signal remains relatively consistent across windows.

XGBoost Performance (Test Set)

Window	Threshold	Precision	Recall	F1
24h	0.78	0.125	0.132	0.128
48h	0.82	0.127	0.106	0.116
72h	0.78	0.127	0.135	0.131

6.1.1 Key Observations

- **72-hour window achieves the highest overall F1 score:** This suggests that expanding the prediction horizon slightly improves model performance, though differences across windows are modest.



- **Performance is broadly consistent across prediction windows:** Precision, recall, and F1 scores remain similar across 24h, 48h, and 72h settings, indicating that the predictive signal is not highly sensitive to the exact window definition.
- **Threshold behavior differs by model type:** Tree-based models (XGBoost, Random Forest) require high classification thresholds (~ 0.75 – 0.82), while Logistic Regression requires a much lower threshold (~ 0.25 – 0.30), reflecting differences in probability calibration.
- **XGBoost consistently outperforms alternative models:** Logistic Regression achieves high recall but impractically low precision, while Random Forest shows limited sensitivity to failure events. This highlights the importance of nonlinear modeling for capturing complex temporal relationships.

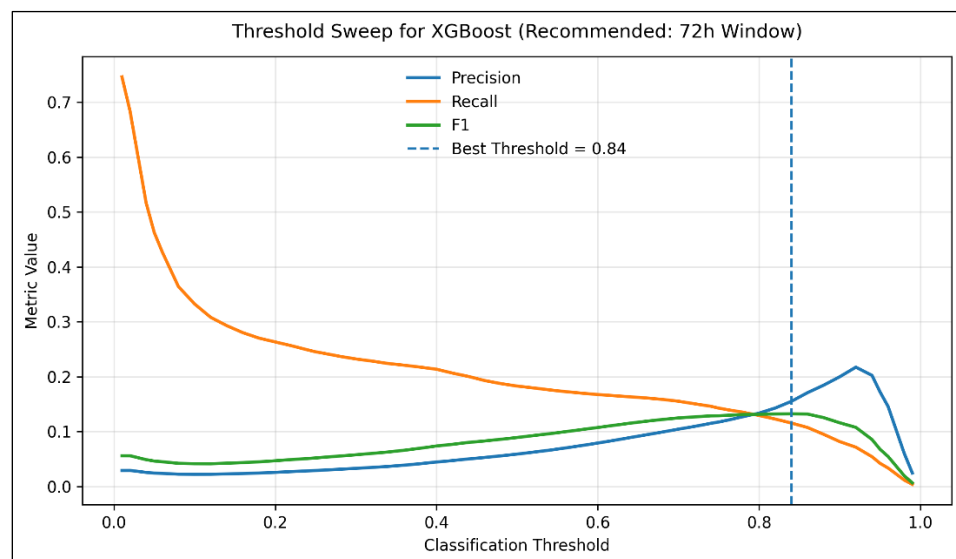
The following table contextualizes model differences.

Comparison Across Models (72h Example)

Model	Threshold	Precision	Recall	F1
XGBoost	0.78	0.127	0.135	0.131
RF	0.76	0.065	0.032	0.043
Logistic	0.28	0.031	0.703	0.059

6.2 Threshold Behavior

Threshold tuning was critical to achieving usable performance, as default thresholds resulted in excessive false positives and poor precision.



- Optimal thresholds are significantly higher than 0.5 for tree-based models
- Strong precision–recall tradeoff is observed across thresholds
- Model probabilities are not well calibrated, requiring threshold adjustment

6.3 Temporal Prediction Behavior

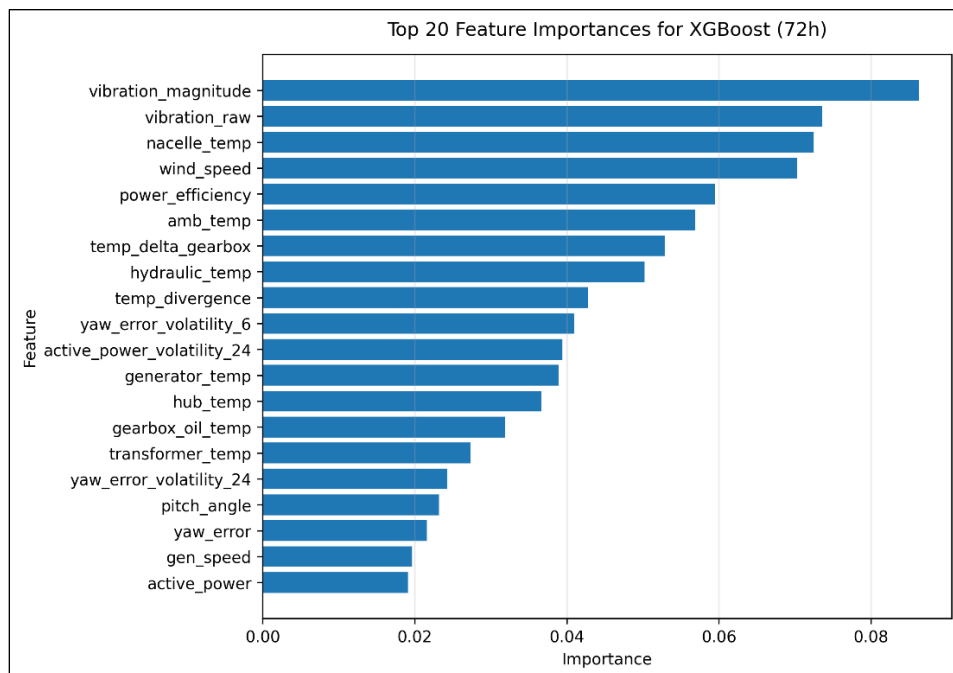
Model predictions exhibit strong temporal structure, with elevated probabilities occurring prior to failure events and persisting over multiple time steps.

- Elevated probabilities consistently precede failure events
- Predictions occur in clusters rather than isolated spikes
- Models often detect signals earlier than the defined prediction window

This behavior suggests that failures are preceded by sustained degradation rather than isolated anomalies.



6.4 Feature Importance



Feature importance analysis indicates that the most influential predictors—particularly vibration-based features (e.g., vibration magnitude and raw vibration), temperature-related signals (e.g., nacelle and gearbox temperature dynamics), and operational variables such as wind speed and power efficiency—align with known physical mechanisms of turbine degradation. The consistency of these features across experiments suggests that the model is capturing meaningful system behavior rather than spurious correlations, providing additional confidence in both the feature engineering approach and the resulting predictions.

6.5 Summary of Results

- XGBoost provides the strongest overall performance among evaluated models
- Model performance is broadly consistent across prediction windows
- Predictive signals are temporally structured and emerge prior to failure events
- Threshold tuning is essential due to probability miscalibration
- Model behavior aligns with known physical degradation processes

These results demonstrate that while failure detection is achievable, aligning predictions with precise and actionable time windows remains the primary challenge.

7 Key Findings and Insights

This section synthesizes the results of exploratory data analysis, feature engineering, and modeling to highlight the most important insights derived from the project.

7.1 Failure Prediction Is a Temporal Degradation Problem

Wind turbine failures are not driven by instantaneous conditions, but by gradual changes in system behavior over time. EDA and model performance both indicate that degradation manifests through increasing vibration, thermal accumulation, and instability in operational signals.

As a result, predictive performance depends on capturing how signals evolve over time rather than relying on static sensor values. This explains the effectiveness of rolling, lag, and rate-of-change features.



7.2 Physically Meaningful Signals Drive Model Performance

Feature importance analysis consistently shows that the most influential predictors align with known physical mechanisms of turbine degradation, including vibration, temperature dynamics, and operational instability.

The dominance of these features across all experiments indicates that the model is learning meaningful system behavior rather than spurious correlations. This strengthens confidence in both the feature engineering approach and the validity of the model outputs.

7.3 Failure Signals Are Diffuse Over Time

Model performance across 24-hour, 48-hour, and 72-hour prediction windows is broadly consistent, with only modest differences in precision, recall, and F1 score. This suggests that failure-related signals are not confined to a sharply defined time window, but instead emerge gradually over an extended period.

This is further supported by temporal prediction behavior, where elevated probabilities often appear well before the defined prediction horizon and persist across multiple time steps.

This indicates that the model is better suited to detecting ongoing system degradation than predicting precise failure timing.

7.4 Prediction Window Defines the Framing, Not the Signal

While the choice of prediction window affects labeling and evaluation, it does not fundamentally change the underlying signal being learned. The similarity in model performance across windows suggests that the same degradation patterns are present regardless of whether the horizon is defined as 24, 48, or 72 hours.

This highlights that labeling strategy primarily influences how predictions are interpreted rather than what the model can learn.

7.5 Precision–Recall Tradeoff Reflects Operational Constraints

The results demonstrate a clear tradeoff between detecting failures (recall) and minimizing false positives (precision). High recall is achievable, particularly with models such as Logistic Regression, but comes at the cost of extremely low precision.

This tradeoff is not purely a modeling issue—it reflects real-world operational constraints. Excessive false alarms are impractical for maintenance teams, while missed detections reduce the value of predictive systems.

Effective model performance must therefore be evaluated in terms of operational usability, not just statistical metrics.

7.6 Nonlinear Models Are Better Suited for This Problem

XGBoost consistently outperformed Logistic Regression and Random Forest, indicating that nonlinear models are better able to capture the complex interactions present in the data.

Logistic Regression achieved high recall but lacked precision, while Random Forest showed limited sensitivity to failure events. This contrast highlights the importance of modeling nonlinear relationships and feature interactions in time-series degradation problems.

7.7 Performance Reflects Problem Complexity Rather Than Model Failure

The observed performance metrics may appear low in absolute terms; however, they must be interpreted in the context of the problem. Wind turbine failure prediction is a rare-event classification task characterized by severe class imbalance, noisy sensor data, and temporally diffuse failure signals.

In such settings, achieving even modest predictive performance is meaningful, particularly when models demonstrate the ability to detect failure-related patterns prior to the event. The results show that the model is capturing non-random structure in the data, as evidenced by consistent performance across prediction windows and alignment with physically meaningful features.

Rather than indicating poor model quality, these results reflect the inherent difficulty of predicting rare and complex system failures. This underscores the importance of evaluating performance in terms of practical utility and relative improvement, rather than relying solely on absolute metric values.

In practice, such models are often used as components of broader decision systems, where even modest predictive signals can provide value when combined with domain knowledge and operational constraints.



7.8 Predictions Are Temporally Correlated

Model outputs exhibit strong temporal clustering, with elevated probabilities persisting across consecutive time steps. This behavior reflects sustained abnormal system conditions rather than isolated anomalies.

This suggests that post-processing techniques, such as persistence-based filtering or smoothing, will be necessary to convert raw model outputs into stable and actionable alerts.

7.9 Overall Synthesis

Taken together, these findings indicate that:

- Failure prediction is driven by gradual temporal degradation patterns.
- Models successfully learn physically meaningful relationships.
- The predictive signal is distributed over time rather than tied to a specific moment.
- The primary challenge lies in aligning predictions with operationally actionable time windows.

These insights provide a strong foundation for refining the modeling approach and improving the practical usability of the system.

8 Challenges and Mitigation Strategies

This section outlines the key challenges encountered during the project and the approaches taken to address them. These challenges reflect both the complexity of the data and the inherent difficulties of applying machine learning to real-world time-series systems.

8.1 Extreme Class Imbalance

One of the most significant challenges in this project is the severe class imbalance, with failure events representing only a small fraction of total observations. Early modeling attempts showed that this imbalance caused models to favor the majority class, resulting in poor recall and limited ability to detect failure events.

To address this, class weighting techniques were applied across models, and evaluation metrics were shifted from accuracy to F1 score and Precision-Recall analysis. In addition, expanding the prediction window (e.g., pre_72h) increased the number of positive samples available for training. These adjustments improved the model's ability to detect failures, though precision remains constrained by the rarity of events.

8.2 Temporal Noise and Early Detection Ambiguity

Another key challenge arises from the temporal nature of failure signals. Models frequently identify elevated risk well before the actual failure event, which indicates strong predictive signal but introduces ambiguity in determining when an alert should be triggered.

To better understand this behavior, multiple prediction windows (24h, 48h, and 72h) were evaluated, and threshold optimization was used to balance sensitivity and specificity. While these approaches improved performance, the results show that failure signals are inherently diffuse, making precise timing of predictions difficult.

8.3 Cross-Farm and Cross-Turbine Variability

Significant variability exists across turbines and wind farms, including differences in sensor availability, calibration, and operating conditions. These inconsistencies create challenges for model generalization, as patterns learned from one subset of turbines may not transfer effectively to others.

This challenge was addressed through sensor harmonization and a focus on derived features, such as deltas and rolling statistics, which are more robust to distributional differences. While these steps improved consistency, variability across turbines remains a key factor influencing model performance.

8.4 Data Leakage in Time-Series Modeling

Time-series modeling introduces a high risk of data leakage, particularly when using rolling features or labels defined relative to future events. Without careful design, models can inadvertently incorporate information from the future, leading to overly optimistic performance estimates.



To mitigate this risk, all preprocessing and feature engineering steps were strictly applied using only past and present data. Buffer zones were also introduced around failure events to remove ambiguous transition periods and prevent contamination from near-event signals. Embedding preprocessing within a unified pipeline further ensured proper separation between training and evaluation data.

8.5 Computational and Memory Constraints

The scale of the dataset, combined with the use of rolling and lag-based features, resulted in significant computational and memory demands. Generating temporal features across large time-series datasets increases both processing time and resource usage.

This was addressed through more efficient data processing techniques, selective feature pruning, and a modular pipeline design that supports scalable experimentation. These improvements enabled the system to handle large datasets while maintaining flexibility for further development.

8.6 Overall Impact of Challenges

Taken together, these challenges highlight the complexity of building predictive models for real-world time-series systems. Addressing issues such as class imbalance, temporal ambiguity, and data heterogeneity required iterative refinement of both the data pipeline and modeling approach.

These challenges also reinforce a key theme of the project: that data design decisions—particularly labeling strategy, feature engineering, and evaluation methodology—are as critical as model selection in determining overall performance.

9 Updated Methodology

This section describes key updates to the original project methodology based on insights gained during exploratory analysis and preliminary modeling results. These refinements reflect a shift toward more realistic and operationally aligned modeling decisions.

9.1 Transition from Static to Temporal Feature Engineering

The initial approach emphasized raw sensor values as model inputs. However, exploratory data analysis revealed that failure signals emerge gradually over time rather than appearing as instantaneous anomalies.

As a result, the feature engineering strategy was revised to focus on temporal representations, including rolling statistics, lag features, and rate-of-change metrics. These features allow the model to capture degradation patterns and evolving system behavior, leading to improved predictive performance.

9.2 Refinement of Labeling Strategy

The original methodology assumed a fixed prediction window. During experimentation, multiple labeling strategies (24-hour, 48-hour, and 72-hour windows) were evaluated to better understand how prediction horizon impacts model performance.

Results showed that model performance is relatively consistent across windows, indicating that failure signals are distributed over time rather than confined to a specific boundary. This led to a broader interpretation of the prediction task as identifying degradation rather than predicting exact failure timing.

9.3 Shift from Recall Emphasis to Balanced Precision–Recall Evaluation

The initial proposal emphasized recall as the primary objective, given the high cost of missed failure events. While this remains an important consideration, early results demonstrated that optimizing solely for recall can lead to impractically high false positive rates.

For example, Logistic Regression achieved high recall but extremely low precision, resulting in excessive false alarms. This behavior highlights the limitations of recall-focused optimization in real-world settings.

As a result, the evaluation strategy was updated to prioritize balanced Precision–Recall performance, using F1 score and Precision–Recall analysis as primary metrics. This shift better reflects operational requirements, where both missed failures and excessive false alarms carry significant costs.



9.4 Introduction of Threshold Optimization

The original approach assumed a default classification threshold (e.g., 0.5). However, model outputs exhibited skewed probability distributions due to class imbalance, making default thresholds ineffective.

To address this, a threshold optimization process was introduced, selecting thresholds that maximize F1 score on validation data. This step significantly improved the usability of model predictions and ensured alignment between model outputs and evaluation objectives.

9.5 Integration of Preprocessing into a Unified Pipeline

Initially, preprocessing steps such as scaling and feature transformation were applied independently. This approach was revised to incorporate all preprocessing steps within a unified modeling pipeline.

This change ensures that transformations are learned exclusively from training data and applied consistently across validation and test sets, reducing the risk of data leakage and improving reproducibility.

9.6 Increased Emphasis on Operational Interpretability

As the project progressed, greater emphasis was placed on ensuring that model outputs are interpretable and aligned with real-world decision-making. This includes understanding the physical meaning of key features, analyzing temporal prediction behavior, and evaluating model outputs in the context of actionable maintenance decisions.

This shift reflects a broader transition from purely statistical modeling toward the development of a practical predictive maintenance system.

9.7 Overall Methodological Evolution

Taken together, these updates represent a shift from an initial modeling-focused approach toward a more comprehensive and system-oriented methodology. Key changes include:

- Incorporating temporal feature engineering.
- Refining labeling strategies based on empirical results.
- Balancing precision and recall reflecting operational constraints.
- Introducing threshold optimization for improved usability.

These refinements demonstrate how the methodology has evolved in response to both data characteristics and modeling outcomes, resulting in a more robust and realistic predictive framework.

10 Future Work Plan

Building on the current results, several areas have been identified as potential next steps for further development and refinement. These represent directions that could improve model usability, align predictions more closely with operational needs, and enhance overall system performance. Given project constraints, it is expected that only a subset of these areas would be prioritized, with selection guided by stakeholder input and project objectives.

10.1 Prediction Smoothing and Alert Generation

One potential area of focus is improving the usability of model outputs by reducing noise and false positives. Current predictions exhibit temporal clustering and intermittent spikes, which can make it difficult to translate raw probabilities into actionable alerts.

Future work could explore post-processing techniques such as persistence-based filtering or rolling aggregation of predictions. These approaches aim to require sustained evidence of degradation before triggering an alert, potentially improving precision and reducing alert fatigue. This direction is particularly promising given the observed temporal correlation in model predictions.

10.2 Lead Time and Event-Based Evaluation

Another important direction is the development of evaluation metrics that better reflect operational value. Current metrics focus on point-wise classification performance, which does not fully capture how early or reliably failures are detected.



Future work could incorporate lead-time analysis and event-based evaluation, measuring how far in advance failures are identified and whether each failure event is successfully detected. These metrics would provide a more meaningful assessment of system effectiveness in a real-world maintenance context.

10.3 Model Refinement and Calibration

While XGBoost demonstrates the strongest performance among the models tested, there is opportunity to improve both predictive accuracy and probability calibration. The current need for high classification thresholds suggests that predicted probabilities are not well aligned with true event likelihood.

Future work could include hyperparameter tuning, probability calibration techniques, or exploration of alternative models such as LightGBM or temporal deep learning approaches. These efforts would aim to improve both model performance and interpretability.

10.4 Improving Generalization Across Turbines

Variability across turbines and wind farms remains a key challenge. Differences in sensor behavior, operating conditions, and data distributions can limit the generalizability of learned patterns.

Future work could explore strategies such as farm-specific calibration, domain adaptation, or hybrid modeling approaches that combine global and turbine-specific components. These approaches aim to improve robustness across diverse operating environments.

10.5 Pipeline Optimization and Scalability

As the dataset and feature space grow, computational efficiency becomes increasingly important. While the current pipeline supports large-scale processing, further optimization could improve experimentation speed and support future deployment scenarios.

Potential improvements include optimizing feature generation workflows, improving data processing efficiency, and enhancing the modularity of the pipeline for easier experimentation and maintenance.

10.6 Overall Direction and Prioritization

Collectively, these areas represent a set of candidate directions for continued development rather than a fixed roadmap. In a real-world setting, these options would be presented to stakeholders to determine which areas provide the greatest value given project goals, resource constraints, and operational priorities.

Given the current findings, initial focus would likely be placed on improving prediction stability (e.g., smoothing and alert logic) and refining evaluation methods (e.g., lead-time analysis), as these areas most directly impact the practical usability of the system.

11 AI Use Disclosure

Generative AI tools (ChatGPT) were used throughout this project as a development aid to improve efficiency, code quality, and clarity of written communication. AI assistance included support for drafting and refining documentation (e.g., headers and docstrings), suggesting code structure and organization, and providing guidance on software architecture decisions to support reproducible experiments and improved computational efficiency.

All core ideas, problem formulation, data processing, feature engineering, model development, experimentation, and interpretation of results were conceived and implemented by the author. AI tools were used to accelerate development and reduce errors, but did not replace independent technical work or decision-making.